

MULTI TP10

Siyuan CHEN

Xiru ZHANG

25 septembre 2025

Table des matières

B. Architecture matérielle	3
Question B1	3
C. Contexte d'exécution d'une tâche	3
Question C1	3
Question C2	4
Question C3	4
Question C4	5
Question C5	5
Question C6	5
Question C7	5
Question C8	6
Question C9	6
Question C10	6
D. Création et lancement des tâches	6
Question D1	6
Question D2	7
Question D3	7
Question D4	7
Question D5	7
Question D6	9
Question D7	9
E. Fonctionnement multi-tâches sur mono-processeur	10
Question E1	10
Question E2	11
Question E3	11

Fonctionnement multi-tâches sur multi-processeurs	12
Question F1	12
Question F2	13
Question F3	14
Question F4	14

B. Architecture matérielle

Question B1

```
1 // IOC
2 icu.p_irq_in[0] = signal_irq_ioc;
3 // in : unused
4 for (size_t x=1; x<8; x++)
5 {
6     icu.p_irq_in[x] = signal_irq_false;
7 }
8 // in : TIMER & DMA
9 for (size_t p=0; p<4; p++)
10 {
11     if (p < nprocs)
12     {
13         icu.p_irq_in[8+p] = signal_irq_dma[p];
14         icu.p_irq_in[12+p] = signal_irq_tim[p];
15     }
16     else
17     {
18         icu.p_irq_in[8+p] = signal_irq_false;
19         icu.p_irq_in[12+p] = signal_irq_false;
20     }
21 }
22 // in : TTY
23 for (size_t p=0; p<4; p++)
24 {
25     for (size_t t=0; t<4; t++)
26     {
27         if ((p < nprocs) and (t < 4))
28             icu.p_irq_in[16+p*4+t] = signal_irq_tty_get[p*4+t];
29         else
30             icu.p_irq_in[16+p*4+t] = signal_irq_false;
31     }
32 }
33 // out : procs
34 for (size_t p=0; p<nprocs; p++)
35 {
36     icu.p_irq_out[p] = signal_irq_proc[p];
37 }
38
```

C. Contexte d'exécution d'une tâche

Question C1

Le pseudo-parallélisme par multiplexage temporel sur un seul processeur est un mécanisme qui permet à un seul processeur d'exécuter plusieurs tâches en alternance de manière à simuler une exécution parallèle. L'événement qui provoque la commutation de tâches

est une interruption de TIMER. Le processus de commutation de tâches est comme suit :

1. La tâche T0 est exécutée.
2. Le cœur reçoit une IRQ du TIMER qui interrompt la tâche T0.
3. Le noyau analyse la cause de l'interruption et exécute la routine d'interruption spécifique du TIMER (*ISR*).
La routine d'interruption utilise une fonction de commutation (*_isr_switch*) qui :
 - Élit la prochaine tâche à exécuter (tâche entrante).
 - Sauvegarde l'état de la tâche actuelle (tâche sortante).
 - Réinitialiser le TIMER de ce tâche.
 - Restaure l'état de la tâche entrante dans le contexte du processeur.
4. Après la terminaison de la fonction de commutation, la routine d'interruption, le GIET donne la main au tâche T1.
5. La tâche T1 commence son exécution.

Ce cycle se répète, permettant à toutes les tâches de progresser en partageant le temps du processeur.

Question C2

Le contexte d'une tâche, qui comprend son état au moment de l'interruption, est sauvegardé dans un tableau nommé `_task_context_array`.

La raison pour laquelle il n'est pas nécessaire de sauvegarder les valeurs stockées dans la pile lors d'un changement de contexte est que chaque processeur, et par extension chaque tâche, dispose de sa propre pile dans la mémoire. Cela signifie que l'état de la pile est inhérent à la tâche et que le pointeur de pile (ou stack pointer) est inclus dans les données de contexte sauvegardées dans `_task_context_array`. Ainsi, lorsque la tâche est reprise, la pile est déjà dans l'état où elle a été laissée, permettant à la tâche de continuer à fonctionner comme si elle n'avait jamais été interrompue.

Question C3

1. La variable `proc_id` est obtenue en appelant la fonction `_procid()`, qui retourne l'identifiant du processeur actuel.
2. Le tableau `_task_number_array[proc_id]` contient le nombre de tâches assignées à chaque processeur. Cela est utilisé pour déterminer s'il y a plus d'une tâche à ordonnancer sur le processeur.
3. Le tableau `_current_task_array` stocke l'index de la tâche actuellement en cours d'exécution sur chaque processeur.
4. Le tableau `_task_context_array` stocke le contexte de toutes les tâches.

En conclusion, le GIET mémorise le placement des tâches sur les processeurs à l'aide des tableaux `_task_number_array` et `_current_task_array`, ainsi que des calculs d'index dans le tableau `_task_context_array`. Cela lui permet de suivre quelle tâche est exécutée sur quel processeur,

Question C4

Via le tableau `_current_task_array`. Le GIET identifie la tâche en cours d'exécution en examinant le tableau `_current_task_array` correspondant à la valeur de l'index de l'ID du processeur. Lorsqu'un changement de tâche est nécessaire, il met à jour les valeurs de ce tableau pour refléter la nouvelle tâche active.

Question C5

Les registre généraux doivent être sauvegardé sauf \$0, \$26, \$27.

1. \$0 : C'est un registre qui contient toujours la valeur zéro, donc il n'est pas nécessaire de le sauvegarder car sa valeur est constante et connue.
2. \$26 & \$27 : Ces registres sont souvent utilisés pour des fonctions spéciales liées au co-processeur et ne sont pas toujours nécessaires pour être sauvegardés dans un contexte de changement de contexte standard.

En ce qui concerne les registres système, il est essentiel de sauvegarder :

1. SR : Parce qu'il contient les indicateurs de mode (utilisateur ou noyau), l'état des interruptions et d'autres bits de statut critiques qui contrôlent le comportement du processeur. Le changement de contexte doit restaurer cet état pour que le processus ou la tâche reprenne correctement son exécution.
2. EPC : Il est crucial car il contient l'adresse du prochain instruction à exécuter lorsque l'exécution revient d'une exception. Si le processus ou la tâche est interrompu par une exception, l'EPC permettra de reprendre l'exécution à l'endroit approprié.
3. CR : Ce registre indique la raison pour laquelle une exception s'est produite, ce qui est nécessaire pour une gestion adéquate des exceptions lors de la reprise de l'exécution d'une tâche.

Question C6

Cette fonction reçoit deux arguments représentant les adresses des contextes de tâches, respectivement pour la tâche en cours d'exécution à désordonner et pour la prochaine tâche à ordonnancer.

Ces deux arguments sont récupérés via registres \$4 et \$5.

Il n'y a pas de valeur de retour pour cette fonction car elle sert juste sauvegarder le contexte actuel d'une tâche et de restaurer le contexte d'une autre tâche.

Question C7

Les deux parties de la fonction `_task_switch` sont :

1. Sauvegarder le contexte de la tâche actuelle.
2. Restaurer le contexte de la tâche prochaine.

Les registres généraux qu'elle a droit de modifier sont de registre \$26 et \$27.

Question C8

Les tâches dans un système d'exploitation multi-tâches doivent avoir un contexte d'exécution distinct pour chaque tâche, y compris les registres du processeur.

Ces registres doivent être définis avant qu'une tâche soit exécutée pour la première fois :

1. Le pointeur de pile (\$29) : Il doit pointer vers le sommet de la pile, moins l'espace nécessaire pour les arguments de la fonction `main()`. C'est essentiel pour garantir que la tâche a suffisamment d'espace pour sa pile d'exécution.
2. Le registre de statut (SR) : Avec la valeur `0xFF13`, cela configure le processeur pour permettre les interruptions et pour s'exécuter en mode utilisateur lorsque la tâche reviendra de l'appel système.
3. Le registre de compteur de programme d'exception (EPC) : Doit être initialisé avec l'adresse de la fonction `main()` de la tâche, de sorte que lorsque l'exécution est transférée à cette tâche, elle commence à l'entrée de `main()`.
4. Les registres \$4 à \$7 : Ils sont utilisés pour les arguments de la fonction `main()`.
5. le registre \$31 : qui contient l'adresse de retour de la fonction `_task_switch()` qui en état stable retourne dans `_ctx_switch()` mais qui initialement passe à une instruction `eret` pour sauter directement dans la tâche.

L'ensemble de ces initialisations de registres est critique et nécessite un contrôle précis et direct de l'état du processeur, que seul le langage assembleur peut fournir de manière fiable.

Question C9

La fonction `_task_switch` se branche à l'adresse stockée dans le registre \$31 lorsqu'elle retourne à la fonction appelante.

La case contenant le registre \$31 du tableau de contexte doit être initialisé car le registre \$31 contient l'adresse de retour, ici configurée pour pointer vers l'instruction qui suit immédiatement l'exécution de `_task_switch()`, pour assurer que, lors de son premier démarrage, la tâche commence avec la bonne adresse de retour.

Question C10

la fonction `_ctx_switch` met en œuvre une politique d'ordonnancement de type round-robin (tourniquet).

D. Création et lancement des tâches

Question D1

Il faut initialiser SR, EPC et SP avant l'exécution de première tâche d'un processeur. Selon ce code, le registre SR est initialisé à `0x0000FF13`. Le registre SP est initialisé

selon le numéro du processeur, et son calcul se fait par :

$$SP = \text{seg_stack_base} + (4 \times \text{pid} + 1) \times 64K$$

avec `seg_stack_base = 0x02000000` et `pid` correspondant au numéro du processeur. Le registre EPC est initialisé en fonction du numéro de tâche, et son calcul est effectué par :

$$\text{task_entry}[\text{task_id}]$$

avec `task_entry = 0xbfc003b0` et `task_id` correspondant au numéro de la tâche.

Question D2

Les 4 cases du tableaux de contexte qui doivent obligatoirement être initialisés sont :

1. `context[n][k][SR] <= 0x0000FF13`
2. `context[n][k][EPC] <= task_entry_point[task_id]`
3. `context[n][k][SP] <= T(n,k) stack pointer`
4. `context[n][k][RA] <= ERET address`

Question D3

```
1      # Initializes SR, SP & EPC for task T(p,0)
2      .word 0b000000000000011110001000100000001 # ICU_MASK[0]
3      .word 0b000000000111100000010001000000000 # ICU_MASK[1]
4      .word 0b000001111000000000010001000000000 # ICU_MASK[2]
5      .word 0b111100000000000000010001000000000 # ICU_MASK[3]
6
```

Question D4

C'est pour la gestion des contextes, chaque ISR peut sauvegarder et restaurer le contexte de la tâche correspondante lorsqu'une interruption se produit. Avoir des ISR distincts simplifie la gestion du contexte pour chaque tâche.

Question D5

```
1      # initializes interrupt vector (This is done by all processors
2      )
3      # This must be consistent with the icu_mask_array[] below...
4      la      $26,    _interrupt_vector # interrupt vector address
5      la      $27,    _isr_ioc
6      sw      $27,    0($26)           # _interrupt_vector[0] <= irq_ioc
7      la      $27,    _isr_dma
8      sw      $27,    32($26)          # _interrupt_vector[8] <= irq_dma[0]
9      sw      $27,    36($26)          # _interrupt_vector[9] <= irq_dma[1]
10     sw      $27,    40($26)          # _interrupt_vector[10] <= irq_dma[2]
11     sw      $27,    44($26)          # _interrupt_vector[11] <= irq_dma[3]
```

```

11     la      $27,    _isr_switch
12     sw      $27,    48($26)      # _interrupt_vector[12] <=
irq_timer[0]
13     sw      $27,    52($26)      # _interrupt_vector[13] <=
irq_timer[1]
14     sw      $27,    56($26)      # _interrupt_vector[14] <=
irq_timer[2]
15     sw      $27,    60($26)      # _interrupt_vector[15] <=
irq_timer[3]
16     la      $26,    _interrupt_vector # interrupt vector address
17     la      $27,    _isr_ioc
18     sw      $27,    0($26)      # _interrupt_vector[0] <= irq_ioc
19     la      $27,    _isr_dma
20     sw      $27,    32($26)      # _interrupt_vector[8] <= irq_dma[0]
21     sw      $27,    36($26)      # _interrupt_vector[9] <= irq_dma[1]
22     sw      $27,    40($26)      # _interrupt_vector[10] <= irq_dma[2]
23     sw      $27,    44($26)      # _interrupt_vector[11] <= irq_dma[3]
24     la      $27,    _isr_switch
25     sw      $27,    48($26)      # _interrupt_vector[12] <= irq_timer
[0]
26     sw      $27,    52($26)      # _interrupt_vector[13] <= irq_timer
[1]
27     sw      $27,    56($26)      # _interrupt_vector[14] <= irq_timer
[2]
28     sw      $27,    60($26)      # _interrupt_vector[15] <= irq_timer
[3]
29     la      $27,    _isr_tty_get_task0
30     sw      $27,    64($26)      # _interrupt_vector[16] <= irq_tty
[0,0]
31     la      $27,    _isr_tty_get_task1
32     sw      $27,    68($26)      # _interrupt_vector[17] <= irq_tty
[0,1]
33     la      $27,    _isr_tty_get_task2
34     sw      $27,    72($26)      # _interrupt_vector[18] <= irq_tty
[0,2]
35     la      $27,    _isr_tty_get_task3
36     sw      $27,    76($26)      # _interrupt_vector[19] <= irq_tty
[0,3]
37     la      $27,    _isr_tty_get_task0
38     sw      $27,    80($26)      # _interrupt_vector[20] <= irq_tty
[1,0]
39     la      $27,    _isr_tty_get_task1
40     sw      $27,    84($26)      # _interrupt_vector[21] <= irq_tty
[1,1]
41     la      $27,    _isr_tty_get_task2
42     sw      $27,    88($26)      # _interrupt_vector[22] <= irq_tty
[1,2]
43     la      $27,    _isr_tty_get_task3
44     sw      $27,    92($26)      # _interrupt_vector[23] <= irq_tty
[1,3]
45     la      $27,    _isr_tty_get_task0
46     sw      $27,    96($26)      # _interrupt_vector[24] <= irq_tty
[2,0]
47     la      $27,    _isr_tty_get_task1
48     sw      $27,    100($26)     # _interrupt_vector[25] <= irq_tty

```



```

49 [2,1]
50 la $27, _isr_tty_get_task2
51 sw $27, 104($26) # _interrupt_vector[26] <= irq_tty
52 [2,2]
53 la $27, _isr_tty_get_task3
54 sw $27, 108($26) # _interrupt_vector[27] <= irq_tty
55 [2,3]
56 la $27, _isr_tty_get_task0
57 sw $27, 112($26) # _interrupt_vector[28] <= irq_tty
58 [3,0]
59 la $27, _isr_tty_get_task1
60 sw $27, 116($26) # _interrupt_vector[29] <= irq_tty
61 [3,1]
62 la $27, _isr_tty_get_task2
63 sw $27, 120($26) # _interrupt_vector[30] <= irq_tty
64 [3,2]
65 la $27, _isr_tty_get_task3
66 sw $27, 124($26) # _interrupt_vector[31] <= irq_tty
67 [3,3]

```

Question D6

Dans le fichier `tp10_top.cpp`, la longueur du segment `seg_stack` est `0x00100000`, qui est égal à $4 \times 4 \times 0x00010000$ (64k*16 tâches), donc la taille est suffisante.

Le pointeur de pile de chacune des tâches est initialisé par

$$\&SP = 0x02000000 + (4 \times pid + 1) \times 0x00010000$$

Question D7

En contrôlant la périodicité des changements de contexte par le système d'exploitation, chaque tâche reçoit une quantité définie de temps processeur, appelée quantum. Comme on souhaite exécuter des programmes indépendants les uns des autres, le mécanisme permettant de remplacer une tâche par une autre sur un même processeur doit être un mécanisme pré-emptif, où la commutation de tâche est imposée par le système d'exploitation, qui réquisitionne le processeur, sans consulter le programme en cours d'exécution. De cette façon, chaque tâche est commutée après une courte période de temps et il semble que plusieurs tâches soient exécutées en même temps.

```

1 # configure and start the timer for the context switch
2 sll $17, $10, 4 # $17 <= pid*16
3 la $16, seg_timer_base
4 addu $16, $16, $17 # $16 <= seg_timer_base + 16*pid
5 li $17, 10000
6 sw $17, 8($16) # period <= 10000
7 li $17, 1
8 sw $17, 4($16) # TIMER[pid] start
9

```

E. Fonctionnement multi-tâches sur mono-processeur

Question E1

```
1  # tasks_entry_point[] array initialization (done by all
2  processors)
3  # we define task_entry_point[p,k] <= main[x] for all T(n,k)
4  tasks
5  # that must be launched on the architecture
6  # The number of entries is 4 procs * 4 tasks
7  # It is indexed by task_id = pid*4 + k
8
9  la $7, seg_data_base
10     la $8, tasks_entry_point
11
12     lw $9, 0($7) # seg_data[0]
13     sw $9, 0($8) # set task_entry_point[0]
14     lw $9, 4($7) # seg_data[1]
15     sw $9, 4($8) # set task_entry_point[1]
16     lw $9, 8($7) # seg_data[2]
17     sw $9, 8($8) # set task_entry_point[2]
18     lw $9, 12($7) # seg_data[3]
19     sw $9, 12($8) # set task_entry_point[3]
20     la $9, _exit
21     sw $9, 16($8) # set task_entry_point[4]
22     la $9, _exit
23     sw $9, 20($8) # set task_entry_point[5]
24     la $9, _exit
25     sw $9, 24($8) # set task_entry_point[6]
26     la $9, _exit
27     sw $9, 28($8) # set task_entry_point[7]
28
29     la $9, _exit
30     sw $9, 0($8) # set task_entry_point[8]
31     la $9, _exit
32     sw $9, 4($8) # set task_entry_point[9]
33     la $9, _exit
34     sw $9, 8($8) # set task_entry_point[10]
35     la $9, _exit
36     sw $9, 12($8) # set task_entry_point[11]
37
38     la $9, _exit
39     sw $9, 16($8) # set task_entry_point[12]
40     la $9, _exit
41     sw $9, 20($8) # set task_entry_point[13]
42     la $9, _exit
43     sw $9, 24($8) # set task_entry_point[14]
44     la $9, _exit
45     sw $9, 28($8) # set task_entry_point[15]
46
47     # _task_number_array[] initialization (done by all processors)
48     # we must define the actual number of tasks assigned to each
49     processor
50     # this must be consistent with the task_entry_point array
51     above.
52     la $8, _task_number_array
```

```

47      li    $9, 4
48      sb    $9, 0($8)      # set _task_number_array[0]
49      li    $9, 0
50      sb    $9, 1($8)      # set _task_number_array[1]
51      li    $9, 0
52      sb    $9, 2($8)      # set _task_number_array[2]
53      li    $9, 0
54      sb    $9, 3($8)      # set _task_number_array[3]
55

```

Question E2

Chaque changement de contexte entre les tâches entraîne un coût supplémentaire non négligeable. Ce coût comprend la sauvegarde et la restauration des états des tâches (par exemple, registres, pointeurs de pile), ce qui prend du temps CPU qui n'était pas nécessaire lorsque `display` s'exécutait seule. À part de cela, en multitâche, les tâches peuvent se disputer l'accès à des ressources partagées

Ce sont les raisons que le rapport n'est pas exactement égal mais inférieur à 4.

Question E3

Dans un système multitâche, la valeur du `TICK`, qui définit la période entre deux interruptions du `TIMER`, est cruciale pour déterminer à quelle fréquence les tâches sont interrompues et le contexte est changé entre elles. La modification de cette valeur influence directement le comportement de l'exécution des tâches sur le processeur. Voici les effets observés lorsque la valeur du `TICK` varie :

1. **Lorsque la période est très grande (1 million de cycles) :**
 - **Comportement Observé :** Chaque tâche s'exécute séquentiellement pendant une longue durée sans interruption. Cela signifie que chaque tâche peut continuer son exécution pendant 1 million de cycles de processeur avant qu'une interruption ne force un changement de contexte.
 - **Explication :** Avec une période aussi grande, le système se rapproche d'une exécution non préemptive où chaque tâche peut s'exécuter presque complètement ou complètement selon sa complexité et la durée de ses opérations, avant de passer à la suivante. Cela peut améliorer l'efficacité pour des tâches longues car il y a moins d'interruptions, ce qui réduit l'overhead associé aux changements de contexte. Cependant, cela peut également conduire à une moins bonne réactivité du système envers les tâches interactives ou celles nécessitant une réponse rapide, car une tâche peut monopoliser le CPU pendant une longue période.
2. **Lorsque la période est très petite (500 cycles) :**
 - **Comportement Observé :** Chaque tâche s'exécute très lentement, semblant progresser peu à chaque intervalle de temps alloué.
 - **Explication :** Avec un `TICK` très court, le système opère de nombreux changements de contexte, interrompant fréquemment chaque tâche. Bien que cela puisse augmenter la réactivité du système en permettant à toutes les tâches

de progresser un peu, cela a un coût significatif en termes de performance due à l'overhead élevé des changements de contexte. Chaque changement de contexte requiert la sauvegarde et la restauration des registres, de la pile, et potentiellement d'autres informations d'état, ce qui consomme une quantité considérable de temps processeur. En conséquence, moins de temps est disponible pour l'exécution réelle des tâches, les rendant plus lentes.

Conclusion :

La valeur du TICK dans un système multitâche doit être choisie avec soin pour équilibrer entre efficacité et réactivité. Un TICK trop long pourrait ne pas répondre efficacement aux besoins des systèmes temps réel ou interactifs, tandis qu'un TICK trop court pourrait réduire considérablement l'efficacité globale en augmentant les coûts liés aux changements de contexte. Trouver le bon TICK est donc essentiel pour optimiser les performances et la réactivité du système.

Fonctionnement multi-tâches sur multi-processeurs

Question F1

```

1      # tasks_entry_point[] array initialization (done by all
      processors)
2      # we define task_entry_point[p,k] <= main[x] for all T(n,k)
      tasks
3      # that must be launched on the architecture
4      # The number of entries is 4 procs * 4 tasks
5      # It is indexed by task_id = pid*4 + k
6      la $7, seg_data_base
7      la $8, tasks_entry_point
8
9      lw $9, 0($7)      # seg_data[0]
10     sw $9, 0($8)      # set task_entry_point[0]
11     lw $9, 4($7)      # seg_data[1]
12     sw $9, 4($8)      # set task_entry_point[1]
13     lw $9, _exit      # seg_data[2]
14     sw $9, 8($8)      # set task_entry_point[2]
15     lw $9, _exit      # seg_data[3]
16     sw $9, 12($8)     # set task_entry_point[3]
17
18     lw $9, 8($7)
19     sw $9, 16($8)     # set task_entry_point[4]
20     lw $9, 12($7)
21     sw $9, 20($8)     # set task_entry_point[5]
22     lw $9, _exit
23     sw $9, 24($8)     # set task_entry_point[6]
24     lw $9, _exit
25     sw $9, 28($8)     # set task_entry_point[7]
26
27     la $9, _exit
28     sw $9, 32($8)     # set task_entry_point[8]
29     la $9, _exit
30     sw $9, 36($8)     # set task_entry_point[9]

```

```

31      la    $9, _exit
32      sw    $9, 40($8)      # set task_entry_point[10]
33      la    $9, _exit
34      sw    $9, 44($8)      # set task_entry_point[11]
35
36      la    $9, _exit
37      sw    $9, 48($8)      # set task_entry_point[12]
38      la    $9, _exit
39      sw    $9, 52($8)      # set task_entry_point[13] 2626
40      la    $9, _exit
41      sw    $9, 56($8)      # set task_entry_point[14]
42      la    $9, _exit
43      sw    $9, 60($8)      # set task_entry_point[15]
44
45      # _task_number_array[] initialization (done by all processors)
46      # we must define the actual number of tasks assigned to each
processor
47      # this must be consistent with the task_entry_point array
above.
48      la    $8, _task_number_array
49      li    $9, 2
50      sb    $9, 0($8)      # set _task_number_array[0]
51      li    $9, 2
52      sb    $9, 1($8)      # set _task_number_array[1]
53      li    $9, 0
54      sb    $9, 2($8)      # set _task_number_array[2]
55      li    $9, 0
56      sb    $9, 3($8)      # set _task_number_array[3]
57

```

Question F2

La différence principale entre le fonctionnement mono-processeur et bi-processeur réside dans la manière dont les tâches sont exécutées et gérées au sein du système.

1. Fonctionnement Mono-processeur :

- **Commutation** : Dans un système mono-processeur, toutes les tâches partagent le même processeur. Cela implique que les tâches doivent être exécutées de manière séquentielle, via une commutation de contexte. La commutation de contexte permet au processeur de passer d'une tâche à une autre, en sauvegardant l'état de la tâche actuelle et en chargeant l'état de la tâche suivante. Cette opération entraîne un certain coût en termes de performance dû à l'overhead nécessaire pour sauvegarder et restaurer les états des tâches.

2. Fonctionnement Bi-processeur :

- **Parallélisme** : Avec deux processeurs, le système peut exécuter les tâches en parallèle. Chaque processeur gère une ou plusieurs tâches indépendamment de l'autre, ce qui peut considérablement accroître l'efficacité et la réactivité du système. Le parallélisme réduit le besoin de commutation de contexte fréquente, puisque chaque processeur peut continuer à exécuter ses tâches assignées sans interruption. Cela permet également une meilleure utilisation

des ressources puisque moins de temps processeur est gaspillé en commutations de contexte.

Conclusion :

En passant d'un système mono-processeur à un système bi-processeur, on observe une amélioration notable de la performance globale. Le fonctionnement bi-processeur permet un traitement des tâches plus rapide et plus efficace grâce au parallélisme, réduisant les délais d'exécution et augmentant la capacité du système à gérer simultanément plusieurs tâches. Cela est particulièrement avantageux dans des environnements où les tâches sont nombreuses et les exigences de réponse rapide sont critiques.

Question F3

1. Le GIET utilise une politique d'ordonnancement fixe et statique, où chaque tâche reçoit une tranche de temps égale pour s'exécuter avant qu'une commutation de contexte ne soit forcée. Cette approche ne prend pas en compte la priorité des tâches ni leur importance relative. Cette approche peut conduire à une mauvaise allocation des ressources processeur.
2. La politique d'ordonnancement ne permet pas de garantir des délais de réponse rapides ou des délais d'exécution prévisibles, essentiels dans les applications temps réel.
3. L'approche de commutation de tâches à intervalles fixes peut conduire à un nombre élevé de changements de contexte, surtout si la période est très courte. Chaque changement de contexte implique une certaine surcharge, réduisant ainsi l'efficacité globale du système en consommant du temps CPU qui pourrait être utilisé pour l'exécution des tâches.

Question F4

```
1  # tasks_entry_point[] array initialization (done by all
   processors)
2  # we define task_entry_point[p,k] <= main[x] for all T(n,k)
   tasks
3  # that must be launched on the architecture
4  # The number of entries is 4 procs * 4 tasks
5  # It is indexed by task_id = pid*4 + k
6  la $7, seg_data_base
7      la $8, tasks_entry_point
8
9      lw $9, 0($7)      # seg_data[0]
10     sw $9, 0($8)      # set task_entry_point[0]
11     lw $9, 4($7)      # seg_data[1]
12     sw $9, 4($8)      # set task_entry_point[1]
13     lw $9, 8($7)      # seg_data[2]
14     sw $9, 8($8)      # set task_entry_point[2]
15     lw $9, 12($7)     # seg_data[3]
16     sw $9, 12($8)     # set task_entry_point[3]
17
```

```

18      lw $9, 0($7)
19      sw $9, 16($8)      # set task_entry_point[4]
20      lw $9, 4($7)
21      sw $9, 20($8)      # set task_entry_point[5]
22      lw $9, 8($7)
23      sw $9, 24($8)      # set task_entry_point[6]
24      lw $9, 12($7)
25      sw $9, 28($8)      # set task_entry_point[7]
26
27      lw $9, 0($7)
28      sw $9, 32($8)      # set task_entry_point[8]
29      lw $9, 4($7)
30      sw $9, 36($8)      # set task_entry_point[9]
31      lw $9, 8($7)
32      sw $9, 40($8)      # set task_entry_point[10]
33      lw $9, 12($7)
34      sw $9, 44($8)      # set task_entry_point[11]
35
36      lw $9, 0($7)
37      sw $9, 48($8)      # set task_entry_point[12]
38      lw $9, 4($7)
39      sw $9, 52($8)      # set task_entry_point[13]
40      lw $9, 8($7)
41      sw $9, 56($8)      # set task_entry_point[14]
42      lw $9, 12($7)
43      sw $9, 60($8)      # set task_entry_point[15]
44
45      # _task_number_array[] initialization (done by all
processors)
46      # we must define the actual number of tasks assigned to
each processor
47      # this must be consistent with the task_entry_point array
above.
48      la $8, _task_number_array
49      li $9, 4
50      sb $9, 0($8)      # set _task_number_array[0]
51      li $9, 4
52      sb $9, 1($8)      # set _task_number_array[1]
53      li $9, 4
54      sb $9, 2($8)      # set _task_number_array[2]
55      li $9, 4
56      sb $9, 3($8)      # set _task_number_array[3]
57

```